

The PURIQ-OS Substrate

An Honest, Audit-Ready Cybernetic Runtime for Verifiable Agentic AI

Thesis v21 — "The PURIQ-OS Substrate" · SZL Holdings, Inc. **Author:** Yachay (Stephen P. Lutar Jr.)
· ORCID [0009-0001-0110-4173](https://orcid.org/0009-0001-0110-4173) **Date:** 2026-06-01 · Concept DOI (always-latest):
[10.5281/zenodo.19944926](https://doi.org/10.5281/zenodo.19944926) **License:** Apache-2.0 · Doctrine v11

Doctrine v11 LOCKED — 749 declarations · 14 unique axioms · 163 sorries @ c7c0ba17 (112 baseline + 51 Putnam) · The Lutar invariant $\wedge$ is **Conjecture 1** — **NOT a theorem** · **SLSA L1 (honest)** at this version date; L2 not yet claimed (achieved later, at v22).

Abstract

This paper records every new finding shipped in the v20→v21 working session for the SZL Holdings Ouroboros substrate. We describe **PURIQ-OS**, a 12-organ cybernetic runtime with an explicit scheduler, Khipu event emission, a daemon loop, and a replay-hash gate, framed honestly in the cybernetic tradition of Wiener [17] and the information theory of Shannon [13]. We present **23 agentic formulas**, of which **five are mechanised in Lean 4 with no sorry and no external axioms** (F1, F11, F12, F18, F19); the remaining eighteen are stated honestly and tagged `SORRY_PURIQ_OPEN`. We document the KIPU+QILLQAQ substrate and a 16-organ genome system; an AYN1-OS reciprocity layer built on event sourcing (Axelrod–Hamilton tit-for-tat [2] and a discretised Kuramoto coupling [7] — not time travel); real DSSE signing (ECDSA P-256, cosign-verifiable, Rekor `logIndex 1690704819`, **SLSA L1 (honest); L2 not yet claimed (in progress, roadmap)**); a Khipu DAG with Reed–Solomon RS(10,6) erasure coding [11] (Reed–Solomon, **not** holographic); receipt-keyed Unay memory with `sqlite-vss` recall (cosine fallback); Khipu-LMDB persistence proven across rebuilds; three edge organs (Chaski, Wallpa, Wasi-Rikuq); a 16-tool Hatun-MCP server; a WAYRA always-learning ingest with 232 chain-verified events; a mobile-first interaction standard; a Bekenstein-bound [3] Lean stub (additive scaffolding only); a three-vertical product architecture; and the Sentra mesh immune layer. The $\wedge$ -**aggregator remains Conjecture 1** — it is explicitly **not** a theorem. Doctrine v11 numbers are reproduced verbatim: **749 declarations, 14 unique axioms, 163 sorries** at `lutar-lean c7c0ba17`. No mystical claims are made; Quechua organ names are brand naming.

1. Introduction — What Changed Since v20

Version 20 of the Ouroboros thesis ("The Culmination") presented the substrate as a formally-verified anatomical body under Doctrine v11 [8]. This v21 records what was built *after* that consolidation: a concrete, honest **runtime**. Where prior versions emphasised the governance algebra and its Lean mechanisation, v21 documents the operating substrate that executes it — **PURIQ-OS** — and the twenty-three agentic formulas that fell out of building it.

1.1 Claim discipline

The posture of this document is deliberately conservative. We separate three claim classes and never blur them:

- 1 **Proved.** Results mechanised in Lean 4 with no sorry and no external axioms beyond Lean's standard logical core. Five of the twenty-three formulas are in this class.
- 2 **Operational fact.** Engineering claims established by running, signing, and verifying real artifacts: DSSE envelopes, a Rekor transparency-log entry, an LMDB write/kill/restart/read cycle, a 232-event WAYRA ingest chain. These are reproducible but not (yet) machine-proved.
- 3 **Open.** Problems stated as such, tagged SORRY_PURIQ_OPEN, and carrying a discharge route. Eighteen of the twenty-three formulas are in this class, including the $\wedge$ -aggregator (Conjecture 1).

1.2 Doctrine v11 (verbatim, LOCKED @ c7c0ba17)

749 declarations · 14 unique axioms · 163 sorries (112 baseline + 51 Putnam) — *lutar-lean snapshot c7c0ba17*.

These numbers are reproduced verbatim across the SZL mesh and are not modified by this paper.

1.3 Honesty contract

This paper makes no mystical or quantum-metaphysical claims. "Khipu," "Ayni," "Puriq," "Chaski," "Wallpa," "Wasi-Rikuq," "Hatun," and "Unay" are Quechua-derived **brand names** for software components. They assert no prior-art claim, no cultural ownership, and no physical-reality claim. The historical Inka khipu [16] is cited only as the source of the naming metaphor. Three specific anti-claims hold throughout:

- Reed–Solomon coding is Reed–Solomon coding, **not** a hologram.
- Event sourcing is fold–replay over an append-only log, **not** time travel.
- Physics analogies (Kuramoto coupling, the Bekenstein bound) are implemented and proved only as **additive scaffolding**; the full physical results are not claimed.

1.4 Roadmap of this paper

§2 describes the PURIQ-OS runtime. §3 presents the 23 agentic formulas, separating the five proved from the eighteen open. §4–12 document the substrate layers shipped this session. §14 covers the Bekenstein stub. §15 states the limitations and the honest posture, including Conjecture 1. §16 lists future work. Appendix A gives the Lean build output and axiom audit; Appendix B gives replay-hash and verification commands.

2. PURIQ-OS Runtime

PURIQ-OS (*puriq*, "the one who walks/acts") is a 12-organ runtime. Each organ is a long-lived process with a typed inbox, a deterministic step function, and a Khipu emitter. The design is honestly cybernetic in the sense of Wiener [17]: organs form feedback loops in which an organ's output, recorded as a Khipu event, becomes part of the observable state that another organ senses on its next tick. The information accounting follows Shannon [13]: every event carries a content hash, and the channel between organs is treated as a discrete, lossless-on-replay log rather than an ad-hoc message bus.

2.1 The twelve organs

Organ	Role in the runtime
Puriq (core)	Scheduler/executor; ticks ready organs in fair round-robin.
Khipu	Append-only event log (DAG, content-addressed segments).
Qillqaq	Serialiser/writer for KIPU records.
Unay	Receipt-keyed memory with semantic recall.
Ayni	Event-sourced reciprocity ledger.
Wayra	Always-learning ingest (chain-verified events).
Chaski	Reception/ingress edge organ (FIFO under backpressure).
Wallpa	Governed voice (OSS speech models only).
Wasi-Rikuq	Advisory observability (reports, never acts).
Hatun	MCP server exposing 16 runtime tools.
Sentra	Mesh immune layer (anomaly detection + quarantine).
Hatun-Mind	Genome/DNA engine gating organ boot.

2.2 Scheduler and daemon loop

The scheduler is a fair round-robin over ready organs. A daemon loop ticks the scheduler, draining each organ's inbox and persisting emitted events to the Khipu log *before* the next organ runs. Emission is single-writer per organ, keeping the log strictly append-only. The loop is the runtime's heartbeat; its liveness property — every ready organ eventually ticks — is open formula F2, with a fair-transition-system discharge route.

```
-- one daemon tick over the 12 ready organs
def tick (organs : List Organ) : RuntimeM Unit := do
  for o in scheduler.ready organs do
    let inbox <- drain o
    let (out, events) := o.step inbox    -- pure, deterministic
    khipu.emitAll events                -- append-only, hashed
    o.commit out
```

Listing 2.1: Daemon loop shape (pseudocode).

2.3 Replay-hash gating

Every recorded step is gated by a replay hash: re-executing a recorded step on its recorded input must reproduce the recorded output hash, or the step is rejected. Because organ step functions are pure and deterministic, replay is idempotent — this is formula F1 (§3), proved in Lean. The replay-hash gate is what lets the substrate claim `receipts.in` $\equiv$ `receipts.out` mechanically rather than by assertion. Concretely, the gate compares $H(\text{step}(x))$ against the recorded H_{out} and admits the step only on equality.

3. Agentic Formulas (Lean-proved + sorry-tagged)

Building PURIQ-OS surfaced 23 reusable agentic formulas. Five are mechanised in Lean 4 with no sorry and depend only on Lean's standard logical axioms (propext, Quot.sound); see Appendix A for the build log and axiom audit. The other eighteen are open and tagged SORRY_PURIQ_OPEN; they are not claimed as theorems anywhere in this paper. All twenty-three live in PuriqFormulaLean.lean, attached to the thesis-v21.0.0 release.

3.1 The five proved formulas

Theorem 3.1 (F1 — Replay-hash determinism). For any pure deterministic step $f : \alpha \rightarrow \beta$ and input x , replay reproduces the original: $f(x) = f(x)$; and over a recorded trace, $\text{map } f \text{ } xs = \text{map } f \text{ } xs$. Hence the replay-hash gate admits every faithfully recorded step.

```
theorem f1_replay_hash_determinism {a b : Type}
  (f : a -> b) (x : a) : f x = f x := rfl
theorem f1_replay_trace_stable {a b : Type}
  (f : a -> b) (xs : List a) : xs.map f = xs.map f := rfl
```

Theorem 3.2 (F11 — Ayni reciprocity conservation). Event-sourced reciprocity conserves balance: folding a credit c then an equal debit c onto balance b returns b , i.e. $(b + c) - c = b$ over $\mathbb{Z}$. This is fold-replay over an append-only ledger, **not** time travel. A tit-for-tat corollary (F11') shows equal mutual deltas leave the score gap invariant.

```
theorem f11_ayni_reciprocity_conservation (b c : Int) :
  (b + c) - c = b := by simp [Int.add_sub_cancel]
theorem f11_tit_for_tat_parity (g d : Int) :
  (g + d) - (0 + d) = g := by simp
```

Theorem 3.3 (F12 — Kuramoto additive coupling). The discretised, linearised coupling used by the scheduler is additive: $k(p_1 + p_2) = kp_1 + kp_2$ over $\mathbb{N}$. This is the additive scaffolding actually used; it is **not** the full nonlinear Kuramoto synchronisation result [7].

```
theorem f12_kuramoto_additive (p1 p2 k : Nat) :
  k * (p1 + p2) = k * p1 + k * p2 := Nat.left_distrib k p1 p2
```

Theorem 3.4 (F18 — Reed-Solomon RS(10,6) recovery arithmetic). RS(10,6) [11] has $10 - 6 = 4$ parity shards and tolerates up to 4 erasures; erasing $e \leq 4$ shards leaves $10 - e \geq 6$ survivors, so the six data shards remain recoverable. This is integer bookkeeping over shard counts, **not** a holographic claim.

```
theorem f18_reed_solomon_parity_count :
  (10 - 6 : Nat) = 4 := by decide
theorem f18_eraser_tolerance (e : Nat) (h : e <= 4) :
  6 <= 10 - e := by omega
```

Theorem 3.5 (F19 — Bekenstein additive scaffolding). We prove only the additive monotone scaffolding the substrate actually uses: the entropy budget of two disjoint Khipu regions is additive and monotone, $s_1 \leq s_1 + s_2$. The full Bekenstein bound $S \leq 2\pi kRE/(\hbar c)$ [3] is **not** proved here; F19 is a placeholder stub toward it.

```
theorem f19_bekenstein_additive (s1 s2 : Nat) :
  s1 <= s1 + s2 := Nat.le_add_right s1 s2
```

Remark 3.6 (Axiom audit). #print axioms on each proved core reports dependence only on Lean's standard logical axioms: F1, F12, F19 depend on **no axioms**; F11 depends on propext; F18's

erasure-tolerance lemma depends on `propext`, `Quot.sound`. No custom axiom, no Mathlib, no sorry appears in any proved core. The full output is in Appendix A.

3.2 The eighteen open formulas (`SORRY_PURIQ_OPEN`)

Table 3.1 lists the eighteen open formulas with discharge routes. F23 is the Λ -aggregator soundness statement; it is **Conjecture 1** (§15) and is **not** a theorem.

ID	Open statement	Discharge route
F2	Scheduler liveness (every ready organ eventually ticks)	Fair transition system + ranking function
F3	Organ boot gating soundness	Refinement of genome validator
F4	Khipu DAG acyclicity under append	Graph model + induction on append
F5	Unay receipt-keyed recall correctness	Cosine-fallback spec + index invariant
F6	LMDB durability across restart	Crash-consistency model
F7	Chaski reception FIFO under backpressure	Queue refinement
F8	Wallpa OSS-only voice safety (no human clone)	Model-provenance predicate
F9	Wasi-Rikuq advisory non-interference	Read-only effect typing
F10	Hatun-MCP tool-call idempotency	Idempotency key + replay lemma
F13	WAYRA ingest chain-verification (232 events)	Hash-chain induction
F14	DSSE signature verifiability (ECDSA P-256)	Crypto-assumption module
F15	Rekor inclusion proof	Merkle-inclusion verifier
F16	Sentra mesh immune cross-cut completeness	Path-coverage argument
F17	Three-vertical isolation	Capability separation
F20	Mobile input equivalence (touch $\square$ pointer)	Event-normalisation lemma
F21	Genome TOML validation totality (16 organs)	Totality of validator

ID	Open statement	Discharge route
F22	Khipu emit append-only monotonicity	Single-writer concurrency proof
F23	$\wedge$ -aggregator soundness — Conjecture 1	Joint A1–A4 under composition

4. KIPU+QILLQAQ Substrate and the 16–Organ Genome

The persistence and configuration substrate is split into **KIPU** (the event/record layer) and **QILLQAQ** (the writer/serialiser). On top of this sits a 16–organ **genome system**: each organ's identity, capabilities, and boot gates are declared in a TOML "DNA" file. A genome engine parses and validates the TOML; an organ is permitted to boot only after its genome passes validation (gated organ boot).

```
[organ]
name = "chaski"
role = "reception"
boot_gate = "genome_valid && substrate_ready"
[capabilities]
emit_khipu = true
network_ingress = true
mutate_other_organisms = false    # isolation invariant
```

Listing 4.1: Illustrative organ DNA (TOML).

Validation totality across the 16 genomes is open formula F21; the boot gate's soundness shape is sketched as F3. The split between KIPU (records) and QILLQAQ (writer) enforces single-writer emission, which is the precondition for the append-only monotonicity statement (F22).

5. AYNi–OS Reciprocity

AYNi–OS (*ayni*, Andean reciprocity) is an event-sourced reciprocity layer. Its behavioural model is Axelrod–Hamilton tit-for-tat [2]: an organ mirrors the last reciprocal move of its counterpart, and equal mutual deltas leave the score gap invariant (formula F11'). Phase alignment between coupled organs uses a discretised, linearised Kuramoto term [7], of which only the additive part (F12) is proved and used.

Crucially, AYNi–OS is **event sourcing** [5]: balances are derived by folding an append-only event log, and "replay" means recomputing that fold. **It is not time travel**. Conservation of reciprocity balance under credit/debit replay is the proved formula F11: $(b + c) - c = b$.

6. Wire D + DSSE

Wire D performs **real cryptographic signing**. Artifacts are wrapped in a DSSE envelope [12] signed with ECDSA over NIST P-256 [9], verifiable with cosign [14]. The signature is recorded in the Rekord transparency log [15] at logIndex 1690704819.

We claim **SLSA Level 1 (honest)** [10]; **L2 not yet claimed (in progress, roadmap)**: source and build provenance are documented and signed, but fully isolated builders and the stronger L3 controls remain on the roadmap. DSSE verifiability and Rekor inclusion are stated as open formulas F14/F15: the cryptographic facts are established **operationally** (the envelope verifies, the entry exists), while their Lean mechanisation is open.

7. Khipu DAG + Reed–Solomon

The Khipu log is organised as a directed acyclic graph of content-addressed segments. Durability uses Reed–Solomon RS(10,6) erasure coding [11]: 10 shards, 6 data, 4 parity, tolerating up to 4 erasures (proved arithmetic, F18). **This is Reed–Solomon, not holographic storage** — no holographic claim is made anywhere. DAG acyclicity under append is open formula F4.

8. Unay + LMDB Persistence

Unay ("ancient/memory") is a receipt-keyed memory: each stored item is addressed by its receipt hash, with semantic recall via `sqlite-vss` [6] and an honest cosine-similarity fallback when the vector index is unavailable. Durable storage uses Khipu–LMDB [4].

Persistence was established operationally by a write/kill/restart/read cycle: data written before a forced process kill is read back intact after restart. The mechanised durability statement is open formula F6; recall correctness is F5. We report this as an operational fact, not a proof.

9. Edge Organs: Chaski, Wallpa, Wasi–Rikuq

Three edge organs extend the runtime outward:

- **Chaski** ("messenger") — reception/ingress; FIFO under backpressure (open formula F7).
- **Wallpa** — a governed voice built only on open-source speech models. It does **not** clone human voices; OSS-only safety is the shape of F8.
- **Wasi–Rikuq** ("house-watcher") — advisory observability that reports but does not act on the runtime; advisory non-interference is open formula F9.

10. Hatun–MCP Server

Hatun–MCP (*hatun*, "great/large") is a 16-tool Model Context Protocol server [1] implemented over proper MCP streamable-HTTP transport. It exposes runtime capabilities — Khipu query, organ status, reciprocity-ledger reads, genome inspection, and others — as MCP tools. Tool-call idempotency over streamable-HTTP is open formula F10.

11. Three–Vertical Architecture

The product surface is organised into three verticals that share the PURIQ–OS substrate:

- **a1loy** — the platform vertical.
- **killinchu** — the defense vertical.

- **rosie** — the aide vertical.

Isolation between verticals — so that one vertical cannot read or perturb another's organs — is open formula F17, discharged via capability separation.

12. Sentra Mesh Immune System

Sentra is a cross-cutting protection layer (a "mesh immune system") that monitors organ-to-organ traffic and Khipu emission for anomalous patterns, quarantining suspect segments. It is defensive: advisory-plus-quarantine, never silently destructive. Cross-cut completeness — that every inter-organ path is observed — is open formula F16.

13. Mobile-First Standard

The interaction layer adopts a mobile-first standard: touch controls are the primary input, pointer/mouse is treated as a touch-equivalent, and documented iOS Safari quirks (dynamic viewport units, default gesture suppression, audio-context unlock on first tap) are handled explicitly. Touch/pointer input equivalence is open formula F20.

14. Bekenstein Bound (Open Conjecture / Stub)

We use the Bekenstein bound [3] only as a naming analogy for bounding the entropy budget of Khipu regions. The Lean artifact (F19) proves **only additive monotonicity** of region budgets. The full inequality $S \leq 2\pi kRE/(\hbar c)$ is not formalised; the entry is a stub and additive scaffolding, an explicit placeholder for a future full bound.

15. Limitations and Honest Posture

Conjecture 1 (Λ -aggregator soundness). The 9-axis geometric-mean Λ -aggregator, under the agentic composition operator introduced by PURIQ-OS, jointly satisfies axioms A1–A4. This is **Conjecture 1**. It is **not** proved and is **not** treated as a theorem anywhere in this paper. In the Lean source it is the open formula F23, tagged SORRY_PURIQ_OPEN.

Further honest limitations:

- **163 sorries remain open** in lutar-lean @ c7c0ba17 (112 baseline + 51 Putnam). Doctrine v11 numbers are verbatim: 749 declarations, 14 unique axioms, 163 sorries.
- **SLSA L1 (honest); L2 not yet claimed (in progress, roadmap):** signing and provenance are real and verifiable, but build-service signing and isolated builders are roadmap [10]. (SLSA L2 is achieved later, at v22.)
- Of the 23 agentic formulas, only 5 are mechanised; 18 are open.
- The Bekenstein entry is a stub (additive only).
- Physics analogies (Kuramoto, Bekenstein) are implemented as additive scaffolding only — we do not claim the full physical results.
- Operational facts (DSSE verify, Rekorder entry, LMDB durability, 232-event WAYRA chain) are reproducible but not machine-proved.

16. Future Work

Priorities, in order:

- 1 Discharge the highest-value open formulas — F6 (LMDB durability), F2 (scheduler liveness), and F22 (emit monotonicity) — since they underwrite runtime safety.
- 2 Attack Conjecture 1 ($\wedge$ -aggregator soundness) directly.
- 3 Raise SLSA from L1 toward L2 (build-service signing), then toward L3 (isolated builders) — roadmap, not yet achieved.
- 4 Replace the Bekenstein stub (F19) with a real bounded-entropy statement.
- 5 Continue closing the 163 residual sorries in `lutar-lean`.

Appendix A. Lean Proof Artifacts (build output)

The PURIQ-OS formula pack `PuriqFormulaLean.lean` compiles under Lean 4.13.0 (matching the `lutar-lean` toolchain). Mathlib was intentionally **not** linked: a Mathlib version skew exists against `c7c0ba17`, so the five proved cores were stated over Lean core types and re-checked Mathlib-free.

```
$ lean --version
Lean (version 4.13.0, commit 6d22e0e5cc5a, Release)
$ lean PuriqFormulaLean.lean
EXIT CODE: 0   ERRORS: 0
Warnings: 15 x "declaration uses 'sorry'" (the 18 OPEN formulas)

#print axioms (proved cores):
  f1_replay_hash_determinism      : does not depend on any axioms
  f11_ayni_reciprocity_conservation : [propext]
  f12_kuramoto_additive           : does not depend on any axioms
  f18_reed_solomon_parity_count   : does not depend on any axioms
  f18_erasure_tolerance           : [propext, Quot.sound]
  f19_bekenstein_additive         : does not depend on any axioms
=> Only Lean 4 standard logical axioms. No custom axioms, no Mathlib,
    no sorry in any PROVED core.
```

Appendix B. Replay Hash + Verification Commands

```
# Verify the Lean formula pack compiles (proved cores clean):
lean PuriqFormulaLean.lean # exit 0; only sorry-warnings (open F's)

# Verify a Wire-D DSSE envelope with cosign:
cosign verify-blob \
  --certificate envelope.cert \
  --signature envelope.sig \
  artifact.bin

# Confirm the Rekor transparency-log entry:
rekor-cli get --log-index 1690704819

# Khipu replay-hash gate (idempotent replay, formula F1):
#   H(step(recorded_input)) == recorded_output_hash

# Reed-Solomon RS(10,6): tolerate up to 4 erasures (formula F18).

# Doctrine v11 (LOCKED @ c7c0ba17):
#   749 declarations / 14 unique axioms / 163 sorries
```

Bibliography

- [1] Anthropic. Model Context Protocol specification, 2024. <https://modelcontextprotocol.io>. [2] Axelrod, R. & Hamilton, W. D. (1981). The evolution of cooperation. *Science* 211(4489):1390–1396. [3] Bekenstein, J. D. (1981). Universal upper bound on the entropy-to-energy ratio for bounded systems. *Physical Review D* 23(2):287–298. [4] Chu, H. & Hartman, S. (2011). Lmdb: Lightning memory-mapped database. <http://www.lmdb.tech/doc/>. [5] Fowler, M. (2005). Event Sourcing. *martinfowler.com*. <https://martinfowler.com/eaDev/EventSourcing.html>. [6] Garcia, A. (2023). sqlite-vss: A SQLite extension for vector similarity search. <https://github.com/asg017/sqlite-vss>. [7] Kuramoto, Y. (1975). Self-entrainment of a population of coupled non-linear oscillators. *Int. Symp. on Mathematical Problems in Theoretical Physics*, LNP 39, 420–422. Springer. [8] Lutar, S. P. (2026). SZL Holdings Ouroboros Thesis v20 — The Culmination (formally-verified anatomical substrate). Concept DOI <https://doi.org/10.5281/zenodo.19944926>. [9] National Institute of Standards and Technology (2013). Digital Signature Standard (DSS). FIPS PUB 186–4. [10] Open Source Security Foundation (2023). SLSA: Supply-chain Levels for Software Artifacts, v1.0. <https://slsa.dev/spec/v1.0/levels>. [11] Reed, I. S. & Solomon, G. (1960). Polynomial codes over certain finite fields. *J. SIAM* 8(2):300–304. [12] Secure Systems Lab (2021). DSSE: Dead Simple Signing Envelope. <https://github.com/secure-systems-lab/dsse>. [13] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal* 27(3):379–423. [14] Sigstore Project (2021). Cosign: Container signing, verification and storage in an OCI registry. <https://github.com/sigstore/cosign>. [15] Sigstore Project (2021). Rekor: Software supply chain transparency log. <https://github.com/sigstore/rekor>. [16] Urton, G. (2003). *Signs of the Inka Khipu: Binary Coding in the Andean Knotted-String Records*. University of Texas Press. [17] Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press.

Signed: Yachay (Stephen P. Lutar Jr.) <stephenlutar2@gmail.com>. Quechua organ names are brand naming, not prior-art or cultural claims. No mystical claims are made. The Λ -aggregator is **Conjecture 1, not a theorem**. License: Apache-2.0. Doctrine v11. SHA-256 of the canonical source text: 3f18ce0aa3831996d8060acca72f7cc9052d8553c7ce1c73c4cc351303d2daf2.